

A Midterm Progress Report
on
HIRIX: AI RESUME SCREENING SYSTEM

**Submitted in partial fulfillment of the requirements for the award of
the degree of**

BACHELOR OF TECHNOLOGY

Computer Science and Engineering

SUBMITTED BY

AMANJOT SINGH (2203397)

UNDER THE GUIDANCE OF

Prof. PreetKamal Singh

Prof. Jaswant Singh

Prof .Palak Sood

June,2026



Department of Computer Science and Engineering
GURU NANAK DEV ENGINEERING COLLEGE,
LUDHIANA

LIST OF FIGURES

Fig. No.	Figure Description	Page No.
4.1	System Architecture of Hirix	15
4.2	Data Flow Diagram	16
4.3	Chronological Execution Sequence	17
6.1	Home Page and Sidebar Configuration	27
6.2	Job Description Input Interface	28
6.3	Resume Upload Interface	28
6.4	Candidate Ranking Dashboard	29
6.5	Resume Analysis Interface	29

LIST OF TABLES

Table. No.	Table Description	Page No.
5.1	Test Cases for Text Extraction Module	21
5.2	Test Cases for Feature Extraction Module	22
5.3	Test Cases for Job Description Processor	22
5.4	Test Cases for Matching Engine	23
5.5	Test Cases for Ranking Module	24

TABLE OF CONTENTS

Contents	Page No.
List of Figures	i
List of Tables	ii
Table of Contents	iii
Chapter 1: Introduction	1
1.1 Introduction to Project	1
1.2 Project Category	2
1.3 Objectives of project	2
Chapter 2: System Requirements	4
2.1 System Requirements	4
2.2 Software Requirements	4
2.3 Functional Requirements	5
2.4 Hardware Requirements	6
Chapter 3: Software Requirement Analysis	7
3.1 Problem Definition	7
3.2 Software Requirement Specification(SRS)	7
3.3 Module Breakdown	10
Chapter 4: Software Design	14
4.1 Detailed Design	14
4.2 Software Design	14
Chapter 5: Testing Module	18
5.1 Testing Technique	18
5.2 Test Cases	21
Chapter 6: Performance and Results	25
6.1 Performance of the Project	25
6.2 Outputs	27
References	31

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

Recruitment teams often receive a large number of resumes for a single job opening, making the initial screening process time-consuming and labor-intensive. Manually reviewing every resume can lead to delays and inconsistencies in candidate selection. To address this challenge, **Hirix** is being developed as an AI-powered resume screening system that automates the evaluation and ranking of candidates based on their suitability for a given job description.

Hirix is built using Python and leverages Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyze resumes and identify the most relevant candidates. The system accepts PDF resumes along with a job description as input and follows a structured screening pipeline to assess candidate-job compatibility. Its primary goal is to reduce manual effort, improve screening efficiency, and support recruiters in making faster and more informed hiring decisions.

The screening process begins with a **Coarse Filter** stage, where the system performs a preliminary analysis of the extracted resume text. During this stage, keywords, technical skills, and important job-related terms are compared with the requirements specified in the job description. Since this operation is computationally inexpensive, it allows the system to quickly eliminate resumes that have little or no relevance to the target position. This helps reduce the number of resumes that require deeper analysis and improves overall system efficiency.

To support different resume formats, including scanned and image-based documents, the project incorporates multiple text extraction technologies. Libraries such as **PyMuPDF**, **pdfplumber**, and **Tesseract OCR** are used to extract textual information from PDF files. Once the text is

extracted, NLP libraries including **spaCy**, **NLTK**, and **sentence-transformers** are employed to preprocess the content, identify important information, and calculate semantic similarity between resumes and job descriptions.

For candidate ranking, Hirix utilizes machine learning techniques to generate relevance scores based on various factors such as skill matching, experience alignment, and educational background. The system can also integrate ensemble learning models such as **Logistic Regression**, **Random Forest**, and **XGBoost** to improve prediction accuracy and provide more reliable candidate rankings. The final output presents candidates in descending order of relevance, enabling recruiters to focus on the most suitable applicants first.

Overall, Hirix aims to provide a scalable, efficient, and intelligent solution for resume screening by combining document processing, NLP, and machine learning techniques into a unified recruitment support system. This project demonstrates the practical application of AI in human resource management and highlights how automation can enhance modern hiring processes.

1.2 Project Category

1. **Application:** Web Application (Streamlit-based interactive dashboard with CLI support)
2. **Domain:** Artificial Intelligence & Natural Language Processing (AI/NLP)
3. **Basis:** Industry-Based (HR & Recruitment Automation)

1.3 Objectives of Project

Project focuses on following objectives:

- To develop an AI – powered resume screening system capable of extracting textual data from resumes.
- To automate the extraction of candidates attributes and perform intelligent ranking and classification based on job requirements .
- To implement a two stage resume screening architecture that combines coarse filtering and detailed evaluation for efficient candidate assessment.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 System Requirements

The Hirix system requires a combination of software tools, libraries, and hardware resources to support resume processing, text extraction, natural language processing, machine learning, and result visualization. The following requirements represent the proposed environment for the successful development and execution of the system.

2.2 Software Requirements

- **Operating System:** The proposed system is designed to operate on modern 64-bit operating systems such as Windows 10/11, Ubuntu Linux, and macOS. These platforms provide compatibility with Python, OCR tools, machine learning libraries, and web-based frameworks used throughout the project. A stable operating system is necessary to ensure smooth execution of resume processing and candidate evaluation tasks.
- **Programming Language:** Python is selected as the primary programming language for developing Hirix. It offers extensive support for Natural Language Processing (NLP), Machine Learning (ML), data manipulation, and web application development. The availability of numerous open-source libraries makes Python suitable for implementing an intelligent resume screening solution.
- **Development Environment:** Visual Studio Code (VS Code) will be used as the primary development environment due to its flexibility, debugging capabilities, and support for Python extensions. Jupyter Notebook may also be utilized during experimentation and model evaluation phases. These tools facilitate efficient coding, testing, and project maintenance.

- **NLP and Machine Learning Libraries:** The project relies on libraries such as spaCy, NLTK, Scikit-learn, and Sentence-Transformers for text preprocessing, feature extraction, semantic analysis, and candidate ranking. These libraries provide pre-built functionalities that help in extracting meaningful information from resumes and job descriptions.
- **PDF Processing and OCR Tools:** To support different resume formats, the system incorporates PyMuPDF and pdfplumber for text extraction from digital PDFs. Tesseract OCR will be used to extract content from scanned or image-based resumes. These tools ensure that the system can process a wide variety of resume documents.
- **Web Framework and Visualization Tools:** The user interface is planned to be developed using Streamlit, which enables the creation of interactive web applications with minimal complexity. Plotly will be used for generating charts and visual analytics that help users understand candidate rankings and screening results more effectively.

2.3 Functional Requirements

- **Resume Upload and Management:** The system should allow recruiters to upload one or multiple resumes in PDF format. Uploaded resumes will be processed automatically and made available for candidate evaluation. The system should support both text-based and scanned documents to ensure flexibility during screening.
- **Job Description Processing:** Users should be able to provide a job description through a text input area or file upload option. The system will analyze the job description and identify important requirements such as skills, educational qualifications, and experience levels that will be used for candidate comparison.
- **Text Extraction and Information Retrieval:** The system should automatically extract textual information from uploaded resumes. After extraction, relevant details such as

technical skills, work experience, educational background, and certifications should be identified and organized for further analysis.

- **Candidate Matching and Ranking:** The extracted resume information should be compared against the job requirements. Based on the degree of similarity and relevance, the system should calculate a matching score and rank candidates accordingly. Higher-ranked candidates should represent better alignment with the job requirements.
- **Result Presentation:** The final screening results should be displayed through an interactive dashboard. Users should be able to view candidate rankings, relevance scores, and key matching details. The system should also provide an option to export results for future reference.

2.4 Hardware Requirements

- **Processor:** A modern multi-core processor is recommended to support resume processing, OCR operations, and machine learning computations. While the system can function on entry-level hardware, better processors will improve overall performance and reduce processing time.
- **Memory (RAM):** The project requires sufficient memory for handling multiple resumes, NLP operations, and model execution. A minimum of 4 GB RAM is required, while 8 GB or more is recommended for smoother performance during batch processing.
- **Storage:** Adequate storage space is required to maintain project files, datasets, trained models, and generated results. A minimum of 5 GB of available storage is recommended.
- **Internet Connectivity:** An internet connection is required during the installation phase for downloading project dependencies and libraries. Once the environment is configured, most resume screening functionalities can operate locally without requiring continuous internet access.

CHAPTER 3

SOFTWARE REQUIREMENT ANALYSIS

3.1 Problem definition

In today's competitive job market, recruiters often receive hundreds of resumes for a single job opening, making the screening process both time-consuming and challenging. Manually reviewing each resume requires significant effort and increases the chances of missing qualified candidates who may be a good fit for the role. The problem becomes even more difficult when resumes are submitted in different formats and styles, making it hard to compare applicants consistently. As a result, organizations need a smarter and more efficient way to identify suitable candidates. The proposed Hirix system aims to address this challenge by automating the resume screening process using Artificial Intelligence, Natural Language Processing, and Machine Learning techniques. By analyzing resumes alongside job descriptions and ranking candidates based on their relevance, the system helps recruiters save time, improve screening accuracy, and make more informed hiring decisions.

3.2 Software Requirement Specification (SRS)

The Software Requirement Specification defines the requirements that guide the design and development of the proposed Hirix resume screening system. These requirements describe the data, functionality, performance, reliability, maintainability, security, and user interface aspects of the system.

3.2.1 Data Requirements

The proposed Hirix system requires candidate resumes and job descriptions as its primary input data. Resumes are expected to be provided in PDF format and may include both text-based and

scanned documents. Job descriptions will serve as the reference for evaluating candidate suitability. During development, publicly available job posting datasets may be utilized for training and testing machine learning models. In addition, the system will rely on NLP resources such as spaCy language models, NLTK corpora, and sentence-transformer models to support text processing and semantic analysis. The generated screening results will be stored in a structured format, while log files will be maintained for monitoring and debugging purposes.

3.2.2 Functional Requirements

The primary objective of Hirix is to automate the resume screening process and assist recruiters in identifying suitable candidates efficiently. The system will allow users to upload resumes and provide job descriptions through an interactive interface. It will extract textual information from resumes, identify important details such as skills, education, certifications, and work experience, and compare them with job requirements. A preliminary filtering stage will be used to eliminate clearly irrelevant resumes before performing detailed analysis. The system is also expected to incorporate semantic matching and machine learning techniques to improve candidate evaluation. Based on the generated relevance scores, candidates will be ranked and presented to the user through a dashboard, with options for exporting the results for further use.

3.2.3 Performance Requirements

The system is expected to provide efficient processing of resumes while maintaining acceptable response times. The proposed two-stage screening pipeline will help reduce computational overhead by filtering irrelevant resumes at an early stage. Hirix should be capable of processing multiple resumes in a single session and should utilize system resources efficiently to ensure smooth operation on standard computing devices. The design should also support future scalability so that larger datasets can be handled with minimal performance degradation.

3.2.4 Dependability Requirements

Reliability is a key requirement of the proposed system. Hirix should continue processing available resumes even if errors occur in individual files. Alternative extraction techniques, such as OCR, should be available when conventional text extraction methods fail. The system should maintain detailed logs of warnings and errors to facilitate troubleshooting and maintenance. Furthermore, it should ensure that extracted information and generated scores remain correctly associated with the corresponding resumes throughout the screening process. Consistent inputs should produce consistent outputs to maintain trust in the system's recommendations.

3.2.5 Maintainability Requirements

The system is intended to follow a modular architecture that separates major functionalities such as text extraction, preprocessing, feature extraction, matching, and ranking. This approach will simplify maintenance, testing, and future enhancements. Configuration settings and system parameters should be managed centrally to allow easy modifications. Proper code documentation and logging practices will also be adopted to improve readability and support future development. Additionally, machine learning models should be replaceable or retrainable without requiring significant changes to the overall application.

3.2.6 Security Requirements

Since resumes contain sensitive personal information, data security must be considered throughout the system design. The proposed application will primarily process data locally to reduce exposure to external services. Uploaded files should be validated before processing to ensure that only supported formats are accepted. Temporary files generated during processing should be removable after completion of the screening process. Access to the system and its

outputs should be restricted to authorized users, thereby helping to protect candidate information and maintain confidentiality.

3.3 Module Breakdown

The proposed Hirix system follows a modular architecture in which each module performs a specific task within the resume screening pipeline. This modular design improves maintainability, scalability, and ease of future enhancements. The major modules of the system are described below.

3.3.1 Resume Input and Document Processing Module

- **Purpose:** Handles the upload and processing of candidate resumes and job descriptions. This module acts as the entry point of the Hirix system.
- **Functionality:**
 - Accepts resumes in PDF format.
 - Supports both text-based and scanned resumes.
 - Extracts text from PDF documents.
 - Uses OCR techniques for image-based resumes.
 - Stores uploaded files temporarily during processing.
- **Technologies:** PyMuPDF, pdfplumber, Tesseract OCR, Pillow.
- **Interdependencies:** Provides extracted text to the Text Preprocessing Module for further analysis.

3.3.2 Text Preprocessing Module

- **Purpose:** Cleans and standardizes extracted text to improve the accuracy of downstream processing.
- **Functionality:**

- Converts text into a uniform format.
- Removes stop words and unnecessary symbols.
- Performs tokenization and normalization.
- Identifies relevant keywords and phrases.
- Prepares data for feature extraction.
- **Technologies:** spaCy, NLTK.
- **Interdependencies:** Receives extracted text from the Document Processing Module and forwards processed text to the Feature Extraction Module.

3.3.3 Feature Extraction Module

- **Purpose:** Extracts important candidate information from resumes in a structured format.
- **Functionality:**
 - Identifies technical and soft skills.
 - Extracts educational qualifications.
 - Detects certifications and achievements.
 - Calculates years of work experience.
 - Organizes extracted information into structured records.
- **Technologies:** spaCy, Regular Expressions (Regex), NLTK.
- **Interdependencies:** Uses preprocessed text and supplies extracted features to the Candidate Matching Module.

3.3.4 Job Description Processing Module

- **Purpose:** Analyzes job descriptions and identifies hiring requirements.
- **Functionality:**
 - Extracts mandatory and preferred skills.
 - Identifies educational requirements.

- Determines required experience levels.
 - Creates a structured representation of job requirements.
 - Supports candidate evaluation and comparison.
- **Technologies:** spaCy, NLTK, Sentence Transformers.
- **Interdependencies:** Provides job requirement features to the Candidate Matching Module.

3.3.5 Candidate Matching Module

- **Purpose:** Evaluates the similarity between candidate profiles and job requirements.
- **Functionality:**
 - Performs keyword-based matching.
 - Calculates skill compatibility scores.
 - Compares educational qualifications.
 - Evaluates experience relevance.
 - Generates overall matching scores.
- **Technologies:** Scikit-learn, Sentence Transformers, TF-IDF.
- **Interdependencies:** Uses outputs from the Feature Extraction and Job Description Processing modules.

3.3.6 Ranking and Classification Module

- **Purpose:** Ranks candidates according to their relevance to the job description.
- **Functionality:**
 - Calculates final relevance scores.
 - Ranks candidates in descending order.
 - Classifies candidates into relevance categories.
 - Provides rejection reasons for low-scoring resumes.

- **Technologies:** Python Scoring Engine, Scikit-learn.
- **Interdependencies:** Receives matching scores from the Candidate Matching Module and forwards results to the Visualization Module.

3.3.7 Machine Learning and Semantic Analysis Module

- **Purpose:** Enhances candidate evaluation through AI and machine learning techniques.
- **Functionality:**
 - Performs semantic similarity analysis.
 - Generates contextual matching scores.
 - Applies machine learning models for relevance prediction.
 - Improves screening accuracy beyond keyword matching.
 - Supports intelligent candidate ranking.
- **Technologies:** XGBoost, Random Forest, Logistic Regression, Sentence Transformers.
- **Interdependencies:** Works alongside the Candidate Matching and Ranking Modules to generate final relevance scores.

3.3.8 Result Visualization Module

- **Purpose:** Presents screening results through an interactive dashboard.
- **Functionality:**
 - Displays ranked candidate lists.
 - Shows relevance scores and classifications.
 - Presents matched and missing skills.
 - Supports result export for further analysis.
- **Technologies:** Streamlit, Plotly, Pandas.
- **Interdependencies:** Receives final rankings and classifications from the Ranking Module and presents them to recruiters through the user interface.

CHAPTER 4

SOFTWARE DESIGN

4.1 Detailed Design

The Hirix AI Resume Screening System is designed to automate and streamline the candidate shortlisting process by leveraging Natural Language Processing (NLP) and Machine Learning (ML) techniques. The proposed system analyzes resumes and job descriptions to identify the most suitable candidates based on their skills, educational qualifications, work experience, and overall relevance to a given job role. To ensure efficiency and scalability, the system follows a modular architecture in which different components handle text extraction, preprocessing, feature extraction, job description analysis, candidate matching, scoring, and ranking. The design also incorporates support for both text-based and scanned PDF resumes through Optical Character Recognition (OCR), enabling broader document compatibility. By combining rule-based evaluation with intelligent data processing techniques, the system aims to reduce manual screening efforts, improve recruitment efficiency, and assist recruiters in making faster and more informed hiring decisions.

4.2 Software Design

4.2.1 System Architecture

The system follows a modular and pipeline-based design that automates the process of resume screening and candidate ranking. The workflow begins with the upload of PDF resumes, which are processed using the PyMuPDF extraction module. If sufficient text cannot be extracted from a resume, the system automatically utilizes a Tesseract OCR fallback mechanism to process scanned or image-based documents. The extracted text then undergoes cleaning,

4.2.2 Data Flow Diagram

The process begins when a candidate's resume in PDF format is provided as input to the system. The resume passes through the Parse and Extract Text module, where textual information is extracted from the document and converted into machine-readable format. The extracted text is then forwarded to the Extract Candidate Attributes module, which identifies and organizes important information such as skills, educational qualifications, certifications, work experience, and contact details into structured feature profiles. Simultaneously, the recruiter provides a job description that is processed by the Parse Job Requirements module to extract required skills, experience criteria, and educational requirements. The structured candidate features and job requirements are then supplied to the Compute Match Scores module, where candidate suitability is evaluated based on multiple matching criteria. The generated scores are subsequently passed to the Rank and Classify module, which ranks candidates according to their relevance and categorizes them based on predefined thresholds. Finally, the screening results are stored in a structured results repository and presented to recruiters through the dashboard, while non-relevant resumes are directed to a separate archive for future reference. As shown in Figure 3.2, the data flows sequentially through each processing stage, ensuring systematic resume evaluation and efficient candidate shortlisting.

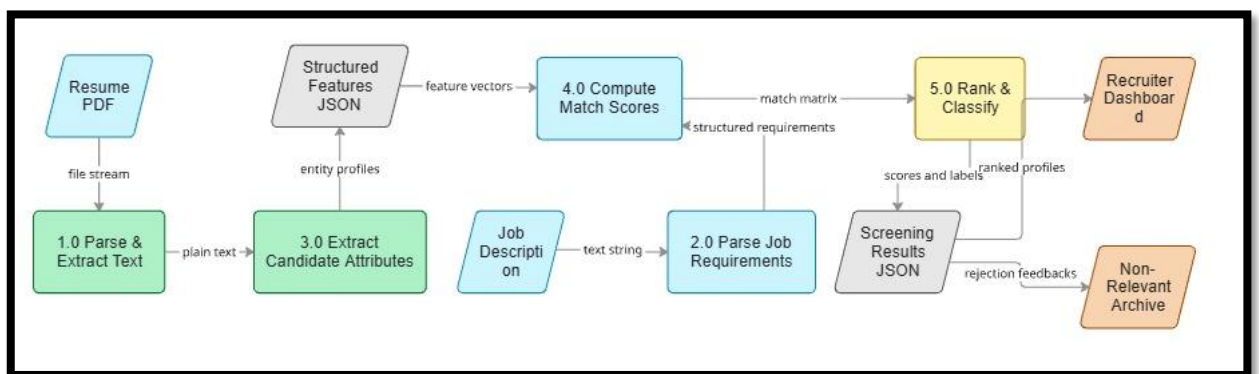


Fig. 4.2 Data Flow Diagram

4.2.3 Sequence Diagram

The process starts when a recruiter uploads resumes and a job description through the user interface. The system orchestrator manages the workflow by extracting key requirements from the job description and candidate information from resumes, including skills, experience, and certifications. The matching engine then compares candidate profiles with job requirements using rule-based and semantic matching techniques. Finally, candidates are ranked based on relevance scores, and the results are displayed to the recruiter through an interactive dashboard.

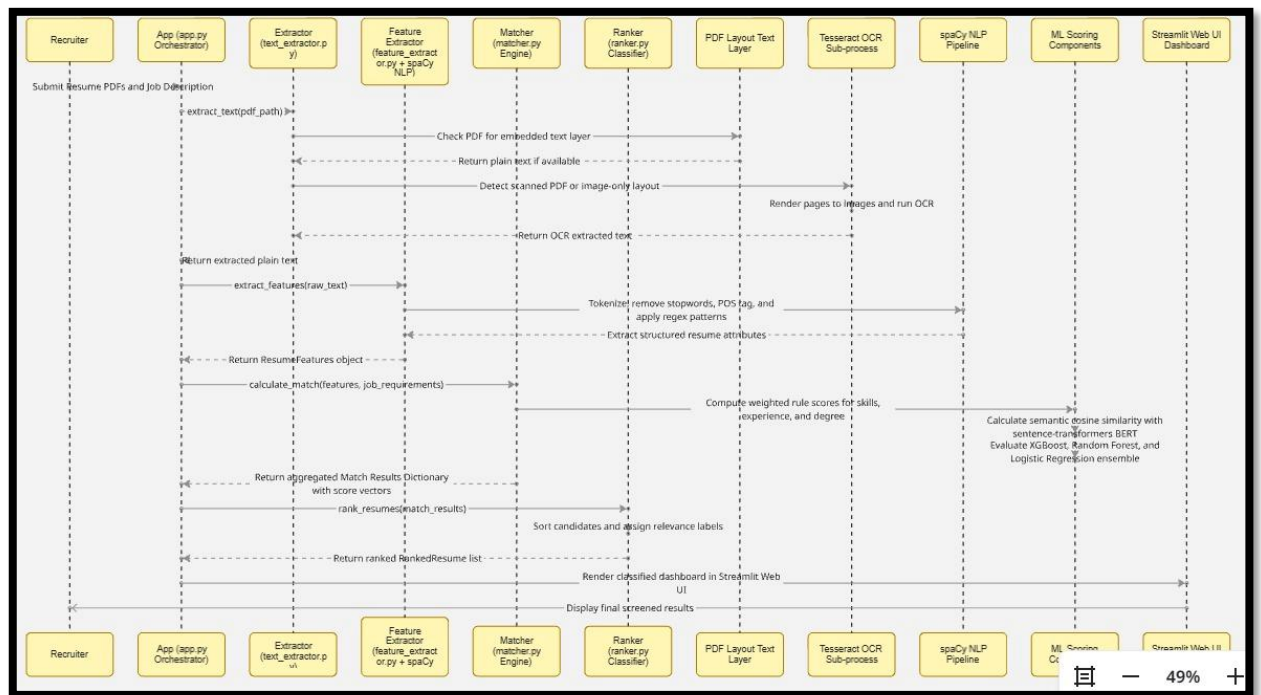


Fig. 4.3 Chronological Execution Sequence

CHAPTER 5

TESTING MODULE

5.1 Testing Techniques

Testing plays an important role in ensuring the reliability, accuracy, and performance of the Hirix AI Resume Screening System. Various testing techniques were applied to verify the correctness of individual modules, integration between system components, user experience, and overall system performance.

5.1.1 Unit Testing

- **Purpose:** Unit testing was performed to verify the functionality of individual modules independently.
- **Modules Tested:**
 - Resume text extraction module
 - Feature extraction module
 - Candidate ranking module
 - Machine learning scoring module
 - User interface components
- **Testing Activities:**
 - Verification of PDF text extraction and OCR functionality.
 - Validation of skill, experience, and education extraction.
 - Testing of similarity calculations and scoring algorithms.
 - Verification of candidate ranking accuracy.
- **Outcome:** All individual modules produced expected outputs and handled invalid inputs appropriately.

5.1.2 Integration Testing

- **Purpose:** Integration testing was conducted to ensure smooth interaction between different modules of the system.
- **Testing Activities:**
 - Integration of resume upload and text extraction modules.
 - Integration of NLP feature extraction and matching engine.
 - Integration of machine learning models with the ranking system.
 - Integration of result visualization and export functionalities.
- **Outcome:** The system successfully processed resumes from upload to final ranking without data loss or inconsistencies.

5.1.3 Usability Testing

- **Purpose:** Usability testing was performed to evaluate the ease of use and effectiveness of the recruiter interface.
- **Testing Activities:**
 - Entering job descriptions.
 - Uploading multiple resumes.
 - Running the screening process.
 - Viewing candidate rankings and analytics.
 - Exporting shortlisted candidates.
- **Outcome:** The interface was found to be intuitive, easy to navigate, and suitable for recruiters with minimal technical knowledge.

5.1.4 Performance Testing

- **Purpose:** Performance testing was conducted to evaluate the system's speed and efficiency while processing large numbers of resumes.

- **Testing Activities:**
 - Measuring resume processing time.
 - Evaluating response time of the ranking engine.
 - Monitoring memory and CPU utilization.
 - Testing batch processing of multiple resumes.
- **Outcome:** The two-stage screening architecture significantly reduced processing time and maintained stable performance under heavy workloads.

5.1.5 Data Integrity and Security Testing

- **Purpose:** This testing ensured that the system handled uploaded files securely and maintained data integrity.
- **Testing Activities:**
 - Validation of uploaded PDF files.
 - Detection of corrupted or unsupported files.
 - Verification of input sanitization.
 - Secure generation of exported reports.
- **Outcome:** The system successfully prevented invalid inputs from affecting processing and maintained the integrity of screening results.

5.1.6 Regression and Smoke Testing

- **Purpose:** Regression and smoke testing were performed to ensure that newly added features did not affect existing functionality.
- **Testing Activities:**
 - Verification of application startup and initialization.
 - Testing core screening functionality after updates.
 - Validation of scoring and ranking consistency.

- Verification of export and visualization features.
- **Outcome:** All critical functionalities operated correctly after modifications, confirming the stability and reliability of the system.

5.2 Test Cases

This section outlines the critical test cases executed to verify the functionality, accuracy, and error handling of the Hirix AI Resume Screening System. The test cases are categorized into UI validation, text extraction, NLP processing, scoring mechanisms, and system outputs.

5.2.1. Test Cases for Text Extraction Module

Table 5.1 Test Cases for Text Extraction Module

TC ID	Test Scenario	Input	Expected Output
TE-01	Extract text from digital PDF	Valid 2-page digital PDF	success=True, method='direct', word_count>100
TE-02	Extract text from scanned PDF	Scanned image-based PDF	success=True, method='ocr'
TE-03	Handle missing file	Non-existent path	success=False, error='File not found'
TE-04	Handle oversized file	15 MB PDF	success=False, error='File too large'
TE-05	Handle corrupt PDF	Truncated file	success=False, error message
TE-06	Handle empty PDF	0-page PDF	success=False, error='No text'
TE-07	Batch extraction	10 valid PDFs	10 success results

5.2.2 Test Cases for Feature Extraction Module

Table 5.2 Test Cases for Feature Extraction Module

TC ID	Test Scenario	Input	Expected Output
FE-01	Extract email	"john@email.com" in resume"	"john@email.com"
FE-02	Extract phone	"Phone: (555) 123-4567"	"(555) 123-4567"
FE-03	Extract Python skill	"Expert in Python and Java"	['java', 'python']
FE-04	Extract experience	"5+ years of experience"	5.0
FE-05	No experience mentioned	"Recent graduate, no exp."	0.0
FE-06	Extract Masters degree	"M.S. in Computer Science"	['m.s.'] or ['master']
FE-07	Extract LinkedIn	"linkedin.com/in/johndoe"	"linkedin.com/in/johndoe"
FE-08	Empty text input	""	Empty ResumeFeatures

5.2.3 Test Cases for Job Description Processor

Table 5.3 Test Cases for Job Description Processor

TC ID	Test Scenario	Input	Expected Output
JD-01	Extract required skills	JD with Python, AWS, ML	['aws', 'machine learning', 'python']
JD-02	Extract experience range	"5-7 years of experience"	min=5.0, max=7.0
JD-03	Detect fresher level	"0-3 years, entry-level"	min=0.0, max=3.0
JD-04	Extract degree	"Bachelor's in CS required"	"bachelors"

TC ID	Test Scenario	Input	Expected Output
JD-05	Detect remote work	"Location: Remote / Hybrid"	remote_allowed=True

5.2.4 Test Cases for Matching Engine

Table 5.4 Test Cases for Matching Engine

TC ID	Test Scenario	Input	Expected Output
MA-01	Perfect skills match	Resume has all required + preferred	skills_score $\approx$ 1.0
MA-02	No skills match	Resume has none of required skills	skills_score $\approx$ 0.0
MA-03	Partial skills match	3 of 5 required skills	$0.3 < \text{skills_score} < 0.8$
MA-04	Fuzzy skill match	"sklearn" vs "scikit-learn"	Matched (ratio $\geq$ 80)
MA-05	Experience within range	5yr resume, 3-8yr required	experience_score = 1.0
MA-06	Experience below range	1yr resume, 5yr required	experience_score $<$ 0.5
MA-07	Degree meets requirement	MS resume, BS required	education_score = 1.0
MA-08	Degree below requirement	BS resume, MS required	education_score $<$ 1.0
MA-09	Semantic similarity (same domain)	ML resume + ML JD	semantic_score $>$ 0.3
MA-10	Semantic similarity (different domain)	HR resume + ML JD	semantic_score $<$ 0.15

5.2.5 Test Cases for Ranking Module

Table 5.5 Test Cases for Ranking Module

TC ID	Test Scenario	Input	Expected Output
RK-01	Rank 10 resumes	10 match results	Sorted DESC by score
RK-02	Classify as relevant	Score = 0.75	is_relevant=True, label="Relevant"
RK-03	Classify as not relevant	Score = 0.25	is_relevant=False, label="Not Relevant"
RK-04	Rejection reasons generated	Score = 0.20, skills=0.15	"Low skills match (15%)"

CHAPTER 6

PERFORMANCE AND RESULTS

6.1 Performance of the Project

The Hirix AI Resume Screening System has made significant progress toward achieving its defined objectives. The development work completed during the current phase focuses on building the foundational components required for automated resume screening, including resume parsing, text extraction, NLP-based feature extraction, candidate matching, and ranking mechanisms. The following progress report evaluates the status of each project objective and highlights the achievements accomplished so far.

6.1.1 Progress Report

- **Objective 1: To develop an AI-powered resume screening system capable of extracting textual data from resumes.**

- **Progress:** Significant progress has been made in developing the resume ingestion and text extraction component of the system. The proposed solution supports PDF resumes and utilizes document parsing techniques to extract textual information. To improve compatibility with different resume formats, an OCR-based fallback mechanism has also been incorporated for handling scanned or image-based resumes. Text cleaning and normalization techniques have been implemented to remove unwanted characters and improve the quality of extracted content.
- **Achievement:** The system is capable of converting resumes into machine-readable text, creating a reliable foundation for subsequent NLP processing and candidate analysis. Support for both text-based and scanned resumes increases the robustness and usability of the screening platform.

• **Objective 2: To automate the extraction of candidate attributes and perform intelligent ranking and classification based on job requirements.**

- **Progress:** An NLP-based processing pipeline has been designed to identify and extract important candidate attributes such as technical skills, educational qualifications, certifications, and work experience from resume content. In parallel, a job description processing module has been developed to identify required skills, preferred qualifications, and experience requirements. The extracted information is then used for candidate-job matching, relevance scoring, and preliminary classification.
- **Achievement:** The system can generate structured candidate profiles and compare them against job requirements to calculate relevance scores. Initial ranking and classification mechanisms have been successfully integrated, enabling automated identification of suitable candidates for a given job role.

• **Objective 3: To implement a two-stage resume screening architecture that combines coarse filtering and detailed evaluation for efficient candidate assessment.**

- **Progress:** The architecture of the proposed system has been designed around a two-stage screening approach. The first stage performs coarse filtering using keyword and skill-based matching to quickly eliminate highly irrelevant resumes. Resumes that pass this stage are forwarded to the second stage, where detailed evaluation is performed using NLP techniques, semantic similarity analysis, and intelligent scoring mechanisms. This design reduces computational overhead while maintaining evaluation accuracy.
- **Achievement:** A structured two-stage screening workflow has been established, enabling efficient processing of large numbers of resumes. The architecture improves screening speed by filtering unsuitable candidates at an early stage.

6.1.2 Overall Progress Status

The Hirix AI Resume Screening System has successfully completed the design and development of its core components, including resume text extraction, NLP-based attribute extraction, candidate matching, and the two-stage screening framework. The system architecture has been finalized and the primary processing modules have been integrated. Current work is focused on refining scoring mechanisms, improving ranking accuracy, enhancing the user interface, and conducting comprehensive testing and evaluation. Overall, the project is progressing according to the planned schedule and is on track toward successful completion.

6.2 Outputs

This section presents the major interfaces of the Hirix AI Resume Screening System along with their functionality and significance in the resume screening process.

6.2.1 Configuration and Input Screens

- **Home Page & Sidebar Configuration:** The home page serves as the entry point of the system. It contains the project title, system description, and a sidebar with configuration settings such as relevance threshold, scoring weights, and pipeline options.

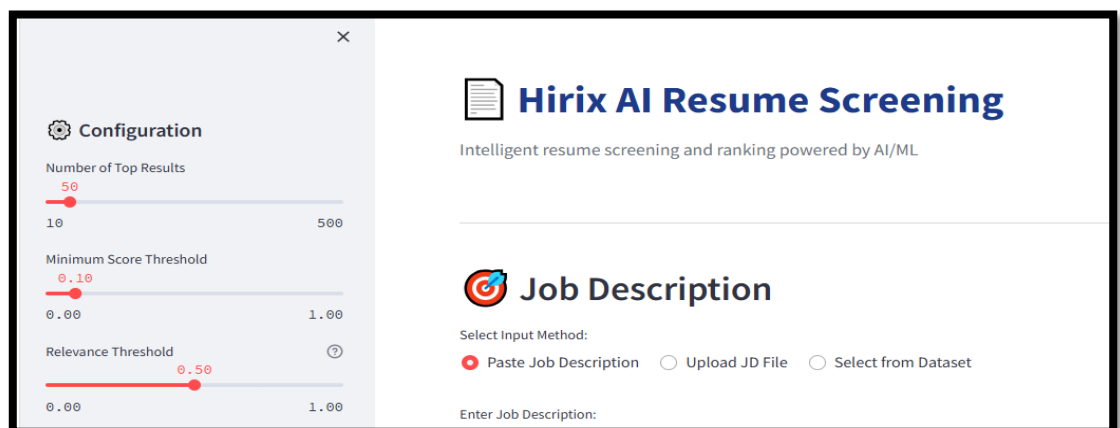


Fig. 6.1 Home Page and Sidebar Configuration

- **Job Description Input Interface** : This screen allows recruiters to provide job descriptions either by entering text manually, uploading a file, or selecting a predefined job posting from the dataset.

Fig. 6.2 Job Description Input Interface

- **Resume Upload Interface:** The resume upload screen supports uploading one or multiple PDF resumes. Users can also select resumes from predefined dataset categories.

Fig. 6.3 Resume Upload Interface

6.2.2 Candidate Evaluation Dashboards

- **Candidate Ranking Dashboard:** This dashboard displays all screened candidates ranked according to their relevance scores. Candidates are categorized as Highly Relevant, Relevant, or Not Relevant.

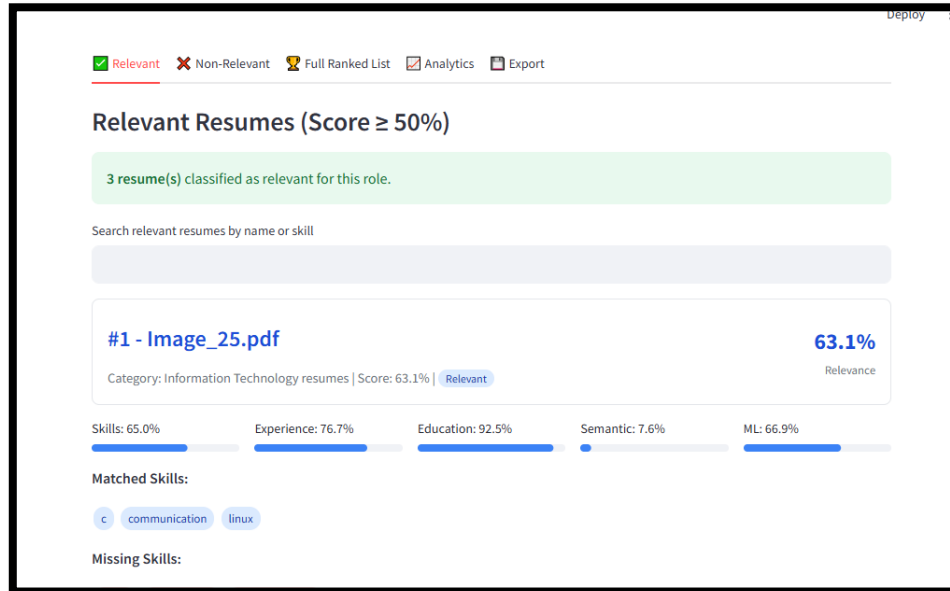


Fig. 6.4 Candidate Ranking Dashboard

- **Resume Analysis Interface:** This screen provides detailed information about an individual candidate, including skills, experience, education details, and score breakdown.

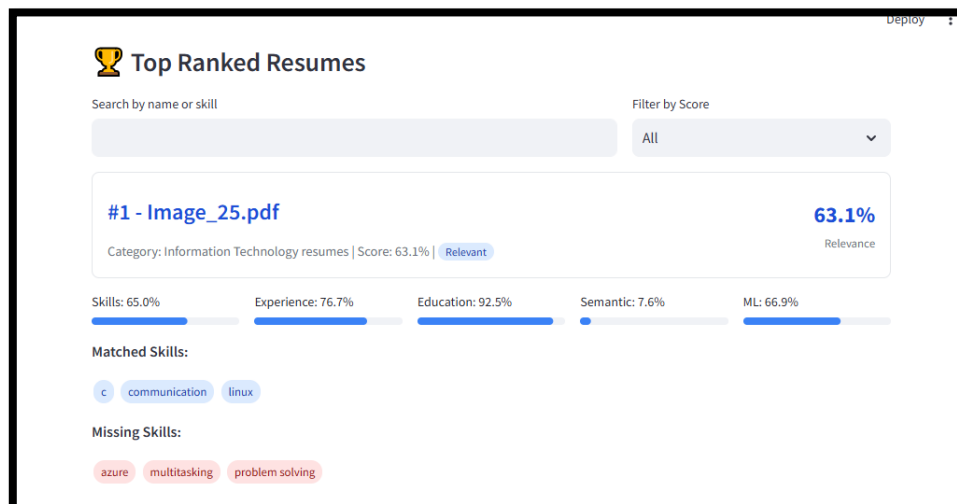


Fig. 6.5 Resume Analysis Interface

References

- [1] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [4] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, China, 2019, pp. 3982–3992.
- [6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] A. Vaswani et al., “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [9] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

- [10] M. Honnibal and I. Montani, “spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” 2017. [Online]. Available: <https://spacy.io>
- [11] Streamlit Inc., “Streamlit – The fastest way to build data applications.” [Online]. Available: <https://streamlit.io>
- [12] R. Smith, “An overview of the Tesseract OCR engine,” in *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, vol. 2, 2007, pp. 629–633.
- [13] PyMuPDF Documentation, “PyMuPDF (fitz) – Python bindings for MuPDF.” [Online]. Available: <https://pymupdf.readthedocs.io>
- [14] NLTK Project, “Natural Language Toolkit (NLTK) Documentation.” [Online]. Available: <https://www.nltk.org>
- [15] RapidFuzz Developers, “RapidFuzz Documentation.” [Online]. Available: <https://rapidfuzz.github.io>
- [16] Plotly Technologies Inc., “Plotly Python Graphing Library Documentation.” [Online]. Available: <https://plotly.com/python>
- [17] Python Software Foundation, “Python 3 Documentation.” [Online]. Available: <https://docs.python.org/3/>